

Homework9 参考答案

Problem 1

文件 5MB，均分 5 段，每段 1MB 在磁盘上连续存放。block 4KB；inode 指针 8B；extent 的 size 域 4B；inode 有 12 个 direct、2 个 indirect、1 个 double indirect。一个 indirect block 能放 $4\text{KB}/8\text{B} = 512$ 个指针。

固定大小 data block 方式

数据块总数 = $5\text{MB} / 4\text{KB} = 1280$ 块，需要 1280 个块指针。

```
12 direct          覆盖 12 块
2 indirect          覆盖  $2 * 512 = 1024$  块   (累计 1036)
double indirect     还差  $1280 - 1036 = 244$  块
    -> 顶层 double-indirect block(512 项, 用 1 项)
        指向 1 个 leaf indirect block(512 项, 用 244 项)
```

实际占用的“定位元数据”空间（indirect 块按整块 4KB 分配）：

```
inode 内指针:  $(12 + 2 + 1) * 8\text{B} = 120\text{ B}$ 
2 个 single indirect block:  $2 * 4\text{KB} = 8\text{ KB}$ 
double indirect 顶层 1 块 + leaf 1 块:  $2 * 4\text{KB} = 8\text{ KB}$ 
合计 =  $16\text{ KB} + 120\text{ B} \approx 16504\text{ B}$ 
```

extent 方式

5 段各自连续，每段用一个 extent = (起始指针 8B + 长度域 4B) = 12B。5 个 extent 共 $5 * 12 = 60\text{B}$ ，可直接放在 inode 中。

结论

固定块方式约 16KB（主要是 indirect 块开销），extent 方式只需 60B。对“大块连续存放”的文件，extent 的定位元数据远小于固定块方式。

Problem 2

文件系统类似 VSFS：superblock、inode bitmap、data bitmap、inodes（12 direct、2 indirect、1 double indirect，指针 4B）、data blocks，block 4KB。计数约定（与课件一致）：缓存初始为空、访问一次即缓存（同一块同一操作内不重复计）；读 superblock、沿途各级目录的 inode 与 data block、相关 bitmap 都各算一次 I/O；indirect/pointer block 单独算一次 I/O；修改文件需更新该文件及其所在目录的时间（再上层目录不更新）；所有修改立刻落盘。

操作 1：向 100KB 的 /foo/bar 后面追加写入 1MB

文件块布局：100KB = 25 块 = 12 个 direct + 13 个在第 1 个 indirect block 中（该 indirect block 已存在）。追加 1MB = 256 块，追加后共 281 块 = 12 direct + 269 个在第 1 个 indirect block（ $269 \leq 1024$ ），只需扩充已有的那个 indirect block，不需要新的 indirect / double indirect block。

读（8 次）：

```
superblock
/ 的 inode
/ 的 data block          (找到 foo)
/foo 的 inode
/foo 的 data block      (找到 bar)
/foo/bar 的 inode
/foo/bar 的第 1 个 indirect block  (要在其中追加指针)
data bitmap             (为 256 个新块找空闲)
```

写（260 次）：

```
256 个新 data block
data bitmap          (标记 256 个块已用，仅 1 块位图)
/foo/bar 的 indirect block (写入新增的 256 个指针)
/foo/bar 的 inode      (更新大小、修改时间)
/foo 的 inode          (更新所在目录的修改时间)
```

合计 I/O = 8 + 260 = 268 次。（不涉及 inode bitmap，因为没有新建 inode；100KB 正好块对齐，无需读旧数据块。）

操作 2：mv /foo/bar /foo/bar2

同目录内改名：只改 /foo 目录数据块里的目录项（bar -> bar2），inode 不搬动。假设 /foo 的数据块有空间容纳新名字，不分配/释放任何块（不动 bitmap）。

读（6 次）：

```
superblock
/ 的 inode
/ 的 data block
/foo 的 inode
/foo 的 data block      (找到 bar 项)
/foo/bar 的 inode       (更新该文件的修改/访问时间)
```

写（3 次）：

/foo 的 data block	(把 bar 改成 bar2)
/foo 的 inode	(目录被修改, 更新时间)
/foo/bar 的 inode	(更新该文件的时间)

合计 I/O = 6 + 3 = 9 次。

说明：以上按课件“读 superblock + 各级目录 inode/data + bitmap 各算一次 I/O”的约定 计数；若约定不计 superblock，则两个操作各减 1（267、8）。改名是否更新被改名文件 自身 inode 的时间也取决于约定，这里按题面“更新该文件及其所在文件夹的时间”都计入。